2.3. Asynchronous Execution

2.3.1. What is Asynchronous Concurrent Execution?

CUDA allows concurrent, or overlapping, execution of multiple tasks, specifically:

- computation on the host
- computation on the device
- memory transfers from the host to the device
- memory transfers from the device to the host
- memory transfers within the memory of a given device
- memory transfers among devices

The concurrency is expressed via an asynchronous interface, where a dispatching function call or kernel launch returns immediately. Asynchronous calls usually return before the dispatched operation has completed and may return before the asynchronous operation has started. The application is then free to perform other tasks at the same time as the originally dispatched operation. When the final results of the initially dispatched operation are needed, the application must perform some form of synchronization to ensure that the operation in question has completed. A typical example of a concurrent execution pattern is the overlapping of host and device memory transfers with computation and thus reducing or eliminating their overhead.

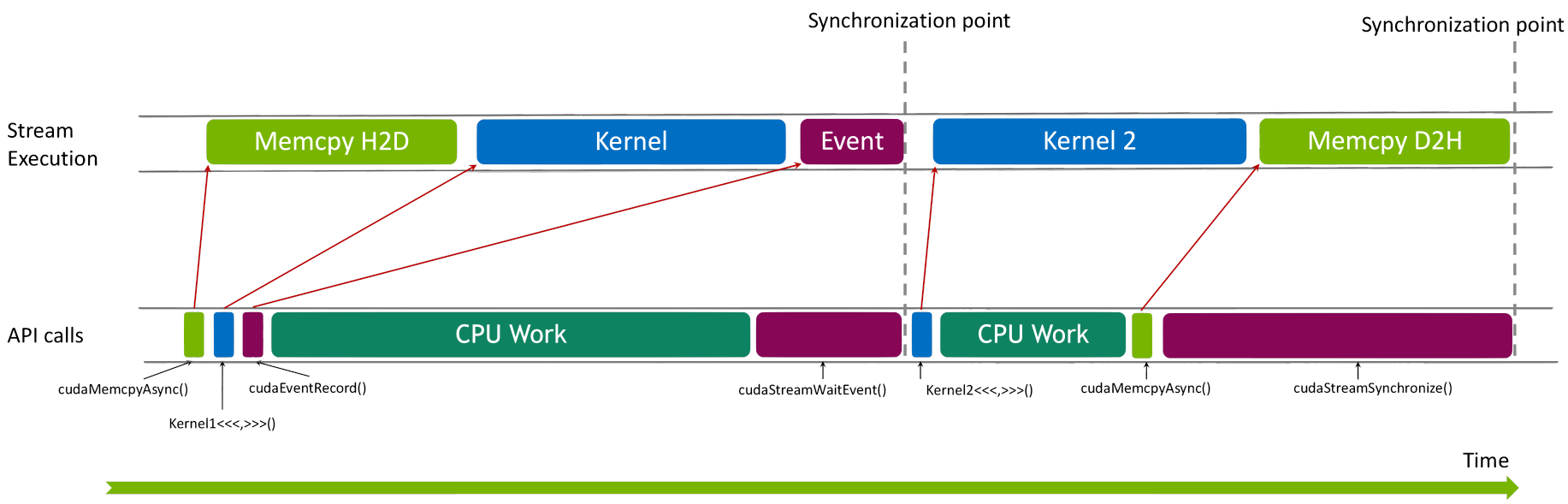

Figure 17: Asynchronous COncurrent Execution with CUDA streams

In general, asynchronous interfaces typically provide three main ways to synchronize with the dispatched operation

- a **blocking approach**, where the application calls a function that blocks, or waits until the operation has completed
- a **non-blocking approach**, or polling approach where the application calls a function that returns immediately and supplies information about the status of the operation
- a **callback approach**, where a pre-registered function is executed when the operation has completed.

While the programming interfaces are asynchronous, the actual ability to carry out various operations concurrently will depend on the version of CUDA and the compute capability of the hardware being used – these details will be left to a later section of this guide (see Compute Capabilities).

In Synchronizing CPU and GPU, the CUDA runtime function `cudaDeviceSynchronize()` was introduced, which is a blocking call which waits for all previously issued work to complete. The reason the `cudaDeviceSynchronize()` call was needed is because the kernel launch is asynchronous and returns immediately. CUDA provides an API for both blocking and non-blocking approaches to synchronization and even supports the use of host-side callback functions.

The core API components for asynchronous execution in CUDA are **CUDA Streams** and **CUDA Events**. In the rest of this section we will explain how these elements can be used to express asynchronous execution in CUDA.

A related topic is that of **CUDA Graphs**, which allow a graph of asynchronous operations to be defined up front, which can then be executed repeatedly with minimal overhead. We cover CUDA Graphs in a very introductory level in section 2.4.9.2 Introduction to CUDA Graphs with Stream Capture, and a more comprehensive discussion is provided in section 4.1 CUDA Graphs.

2.3.2. CUDA Streams

At the most basic level, a CUDA stream is an abstraction which allows the programmer to express a sequence of operations. A stream operates like a work-queue into which programs can add operations, such as memory copies or kernel launches, to be executed in order. Operations at the front of the queue for a given stream are executed and then dequeued allowing the next queued operation to come to the front and to be considered for execution. The order of execution of operations in a stream is sequential and the operations are executed in the order they are enqueued into the stream.

An application may use multiple streams simultaneously. In such cases, the runtime will select a task to execute from the streams that have work available depending on the state of the GPU resources. Streams may be assigned a priority which acts as a hint to the runtime to influence the scheduling, but does not guarantee a specific order of execution.

The API function calls and kernel-launches operating in a stream are asynchronous with respect to the host thread. Applications can synchronize with a stream by waiting for it to be empty of tasks, or they can also synchronize at the device level.

CUDA has a default stream, and operations and kernel launches without a specific stream are queued into this default stream. Code examples which do not specify a stream are using this default stream implicitly. The default stream has some specific semantics which are discussed in subsection Blocking and non-blocking streams and the default stream.

2.3.2.1. Creating and Destroying CUDA Streams

CUDA streams can be created using the `cudaStreamCreate()` function. The function call initializes the stream handle which can be used to identify the stream in subsequent function calls.

```
cudaStream_t stream;        // Stream handle
cudaStreamCreate(&stream);  // Create a new stream

// stream based operations ...

cudaStreamDestroy(stream);  // Destroy the stream
```

If the device is still doing work in stream `stream` when the application calls `cudaStreamDestroy()`, the stream will complete all the work in the stream before being destroyed.

2.3.2.2. Launching Kernels in CUDA Streams

The usual triple-chevron syntax for launching a kernel can also be used to launch a kernel into a specific stream. The stream is specified as an extra parameter to the kernel launch. In the following example the kernel named `kernel` is launched into the stream with handle `stream`, which is of type `cudaStream_t` and has been assumed to have been created previously:

```
kernel<<<grid, block, shared_mem_size, stream>>>(...);
```

The kernel launch is asynchronous and the function call returns immediately. Assuming that the kernel launch is successful, the kernel will execute in the stream `stream` and the application is free to perform other tasks on the CPU or in other streams on the GPU while the kernel is executing.

2.3.2.3. Launching Memory Transfers in CUDA Streams

To launch a memory transfer into a stream, we can use the function **`cudaMemcpyAsync()`**. This function is similar to the **`cudaMemcpy()`** function, but it takes an additional parameter specifying the stream to use for the memory transfer. The function call in the code block below copies `size` bytes from the host memory pointed to by `src` to the device memory pointed to by `dst` in the stream `stream`.

```
// Copy `size` bytes from `src` to `dst` in stream `stream`
cudaMemcpyAsync(dst, src, size, cudaMemcpyHostToDevice, stream);
```

Like other asynchronous function calls, this function call returns immediately, whereas the **`cudaMemcpy()`** function blocks until the memory transfer is complete. In order to access the results of the transfer safely, the application must determine that the operation has completed using some form of synchronization.

Other CUDA memory transfer functions such as **`cudaMemcpy2D()`** also have asynchronous variants.

> **Note**
>
> In order for memory copies involving CPU memory to be carried out asynchronously, the host buffers must be pinned and page-locked. **`cudaMemcpyAsync()`** will function correctly if host memory which is not pinned and page-locked is used, but it will revert to a synchronous behavior which will not overlap with other work. This can inhibit the performance benefits of using asynchronous memory transfers. It is recommended programs use **`cudaMallocHost()`** to allocate buffers which will be used to send or receive data from GPUs.

2.3.2.4. Stream Synchronization

The simplest way to synchronize with a stream is to wait for the stream to be empty of tasks. This can be done in two ways, using the **`cudaStreamSynchronize()`** function or the **`cudaStreamQuery()`** function.

The **`cudaStreamSynchronize()`** function will block until all the work in the stream has completed.

```
// Wait for the stream to be empty of tasks
cudaStreamSynchronize(stream);

// At this point the stream is done
// and we can access the results of stream operations safely
```

If we prefer not to block, but just need a quick check to see if the steam is empty we can use the **`cudaStreamQuery()`** function.

```
// Have a peek at the stream
// returns cudaSuccess if the stream is empty
// returns cudaErrorNotReady if the stream is not empty
cudaError_t status = cudaStreamQuery(stream);

switch (status) {
    case cudaSuccess:
        // The stream is empty
        std::cout << "The stream is empty" << std::endl;
        break;
    case cudaErrorNotReady:
        // The stream is not empty
        std::cout << "The stream is not empty" << std::endl;
        break;
    default:
        // An error occurred - we should handle this
        break;
};
```

2.3.3. CUDA Events

CUDA events are a mechanism for inserting markers into a CUDA stream. They are essentially like tracer particles that can be used to track the progress of tasks in a stream. Imagine launching two kernels into a stream. Without such tracking events, we would only be able to determine whether the stream is empty or not. If we had an operation that was dependent on the output of the first kernel, we would not be able to start that operation safely until we knew the stream was empty by which time both kernels would have completed.

Using CUDA Events we can do better. By enqueuing an event into a stream directly after the first kernel, but before the second kernel, we can wait for this event to come to the front of the stream. Then, we can safely start our dependent operation knowing that the first kernel has completed, but before the second kernel has started. Using CUDA events in this way can build up a graph of dependencies between operations and streams. This graph analogy translates directly into the later discussion of CUDA graphs.

CUDA streams also keep time information which can be used to time kernel launches and memory transfers.

2.3.3.1. Creating and Destroying CUDA Events

CUDA Events can be created and destroyed using the `cudaEventCreate()` and `cudaEventDestroy()` functions.

```
cudaEvent_t event;

// Create the event
cudaEventCreate(&event);

// do some work involving the event

// Once the work is done and the event is no longer needed
// we can destroy the event
cudaEventDestroy(event);
```

The application is responsible for destroying events when they are no longer needed.

2.3.3.2. Inserting Events into CUDA Streams

CUDA Events can be inserted into a stream using the `cudaEventRecord()` function.

```
cudaEvent_t event;
cudaStream_t stream;

// Create the event
cudaEventCreate(&event);

// Insert the event into the stream
cudaEventRecord(event, stream);
```

2.3.3.3. Timing Operations in CUDA Streams

CUDA events can be used to time the execution of various stream operations including kernels. When an event reaches the front of a stream it records a timestamp. By surrounding a kernel in a stream with two events we can get an accurate timing of the duration of the kernel execution as is shown in the code snippet below:

```
cudaStream_t stream;
cudaStreamCreate(&stream);

cudaEvent_t start;
cudaEvent_t stop;

// create the events
cudaEventCreate(&start);
cudaEventCreate(&stop);

 // record the start event
cudaEventRecord(start, stream);

// launch the kernel
kernel<<<grid, block, 0, stream>>>(...);

// record the stop event
cudaEventRecord(stop, stream);

// wait for the stream to complete
// both events will have been triggered
cudaStreamSynchronize(stream);

// get the timing
float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);
std::cout << "Kernel execution time: " << elapsedTime << " ms" << std::endl;

// clean up
cudaEventDestroy(start);
cudaEventDestroy(stop);
cudaStreamDestroy(stream);
```

2.3.3.4. Checking the Status of CUDA Events

Like in the case of checking the status of streams, we can check the status of events in either a blocking or a non-blocking way.

The `cudaEventSynchronize()` function will block until the event has completed. In the code snippet below we launch a kernel into a stream, followed by an event and then by a second kernel. We can use the `cudaEventSynchronize()` function to wait for the event after the first kernel to complete and in principle launch a dependent task immediately, potentially before *kernel2* finishes.

```
cudaEvent_t event;
cudaStream_t stream;

// create the stream
cudaStreamCreate(&stream);

// create the event
cudaEventCreate(&event);

// launch a kernel into the stream
kernel<<<grid, block, 0, stream>>>(...);

// Record the event
cudaEventRecord(event, stream);

// launch a kernel into the stream
kernel2<<<grid, block, 0, stream>>>(...);

// Wait for the event to complete
// Kernel 1 will be  guaranteed to have completed
// and we can launch the dependent task.
cudaEventSynchronize(event);
dependentCPUtask();

// Wait for the stream to be empty
// Kernel 2 is guaranteed to have completed
cudaStreamSynchronize(stream);

// destroy the event
cudaEventDestroy(event);

// destroy the stream
cudaStreamDestroy(stream);
```

CUDA Events can be checked for completion in a non-blocking way using the `cudaEventQuery()` function. In the example below we launch 2 kernels into a stream. The first kernel, *kernel1* generates some data which we would like to copy to the host, however we also have some CPU side work to do. In the code below, we enqueue *kernel1* followed by an event (*event*) and then *kernel2* into stream *stream1*. We then go into a CPU work loop, but occasionally take a peek to see if the event has completed indicating that *kernel1* is done. If so, we launch a host to device copy into stream *stream2*. This approach allows the overlap of the CPU work with the GPU kernel execution and the device to host copy.

```
cudaEvent_t event;
cudaStream_t stream1;
cudaStream_t stream2;

size_t size = LARGE_NUMBER;
float *d_data;

// Create some data
cudaMalloc(&d_data, size);
float *h_data = (float *)malloc(size);

// create the streams
cudaStreamCreate(&stream1);   // Processing stream
cudaStreamCreate(&stream2);   // Copying stream
bool copyStarted = false;

//  create the event
cudaEventCreate(&event);

// launch kernel1 into the stream
kernel1<<<grid, block, 0, stream1>>>(d_data, size);
// enqueue an event following kernel1
cudaEventRecord(event, stream1);

// launch kernel2 into the stream
kernel2<<<grid, block, 0, stream1>>>();

// while the kernels are running do some work on the CPU
// but check if kernel1 has completed because then we will start
// a device to host copy in stream2
while ( not allCPUWorkDone() || not copyStarted ) {
    doNextChunkOfCPUWork();

    // peek to see if kernel 1 has completed
    // if so enqueue a non-blocking copy into stream2
    if ( not copyStarted ) {
        if( cudaEventQuery(event) == cudaSuccess ) {
            cudaMemcpyAsync(h_data, d_data, size, cudaMemcpyDeviceToHost, stream2);
            copyStarted = true;
        }
    }
}

// wait for both streams to be done
cudaStreamSynchronize(stream1);
cudaStreamSynchronize(stream2);

// destroy the event
cudaEventDestroy(event);

// destroy the streams and free the data
cudaStreamDestroy(stream1);
cudaStreamDestroy(stream2);
cudaFree(d_data);
free(h_data);
```

2.3.4. Callback Functions from Streams

CUDA provides a mechanism for launching functions on the host from within a stream. There are currently two functions available for this purpose: `cudaLaunchHostFunc()` and `cudaAddCallback()`. However, `cudaAddCallback()` is slated for deprecation, so applications should use `cudaLaunchHostFunc()`.

Using `cudaLaunchHostFunc()`

The signature of the `cudaLaunchHostFunc()` function is as follows:

```
cudaError_t cudaLaunchHostFunc(cudaStream_t stream, void (*func)(void *), void *data);
```

where

- `stream`: The stream to launch the callback function into.
- `func`: The callback function to launch.
- `data`: A pointer to the data to pass to the callback function.

The host function itself is a simple C function with the signature:

```
void hostFunction(void *data);
```

with the `data` parameter pointing to a user defined data structure which the function can interpret. There are some caveats to keep in mind when using callback functions like this. In particular, the host function may not call any CUDA APIs.

For the purposes of being used with unified memory, the following execution guarantees are provided: - The stream is considered idle for the duration of the function's execution. Thus, for example, the function may always use memory attached to the stream it was enqueued in. - The start of execution of the function has the same effect as synchronizing an event recorded in the same stream immediately prior to the function. It thus synchronizes streams which have been "joined" prior to the function. - Adding device work to any stream does not have the effect of making the stream active until all preceding host functions and stream callbacks have executed. Thus, for example, a function might use global attached memory even if work has been added to another stream, if the work has been ordered behind the function call with an event. - Completion of the function does not cause a stream to become active except as described above. The stream will remain idle if no device work follows the function, and will remain idle across consecutive host functions or stream callbacks without device work in between. Thus, for example, stream synchronization can be done by signaling from a host function at the end of the stream.

2.3.4.1. Using `cudaStreamAddCallback()`

> **Note**
>
> The **`cudaStreamAddCallback()`** function is slated for deprecation and removal and is discussed here for completeness and because it may still appear in existing code. Applications should use or switch to using **`cudaLaunchHostFunc()`**.

The signature of the **`cudaStreamAddCallback()`** function is as follows:

```
cudaError_t cudaStreamAddCallback(cudaStream_t stream, cudaStreamCallback_t callback, void* userData, unsigr
```

where

- `stream`: The stream to launch the callback function into.
- `callback`: The callback function to launch.
- `userData`: A pointer to the data to pass to the callback function.
- `flags`: Currently, this parameter must be 0 for future compatibility.

The signature of the `callback` function is a little different from the case when we used the **`cudaLaunchHostFunc()`** function. In this case the callback function is a C function with the signature:

```
void callbackFunction(cudaStream_t stream, cudaError_t status, void *userData);
```

where the function is now passed

- `stream`: The stream handle from which the callback function was launched.

- `status`: The status of the stream operation that triggered the callback.
- `userData`: A pointer to the data that was passed to the callback function.

In particular the `status` parameter will contain the current error status of the stream, which may have been set by previous operations. Similarly to the **`cudaLaunchHostFunc()`** func case, the stream will not be active and advance to tasks until the host-function has completed, and no CUDA functions may be called from within the callback function.

2.3.4.2. Asynchronous Error Handling

In a cuda stream, errors may originate from any operation in the stream, including for kernel launches and memory transfers. These errors may not be propagated back to the user at run-time until the stream is synchronized, for example, by waiting for an event or calling **`cudaStreamSynchronize()`**. There are two ways to find out about errors which may have occurred in a stream.

- Using the function **`cudaGetLastError()`** - this function returns and clears the last error encountered in any stream in the current context. An immediate second call to cudaGetLastError() would return `cudaSuccess` if no other error occurred between the two calls.
- Using the function **`cudaPeekAtLastError()`** - this function returns the last error in the current context, but does not clear it.

Both of these functions return the error as a value of type `cudaError_t`. Printable names names of the errors can be generated using the functions *cudaGetErrorName()* and *cudaGetErrorString()*.

An example of using these functions is shown below:

Listing 1: Example of using cudaGetLastError() and cudaPeekAtLastError()

```
// Some work occurs in streams.
cudaStreamSynchronize(stream);

// Look at the last error but do not clear it
cudaError_t err = cudaPeekAtLastError();
if (err != cudaSuccess) {
    printf("Error with name: %s\n", cudaGetErrorName(err));
    printf("Error description: %s\n", cudaGetErrorString(err));
}

// Look at the last error and clear it
cudaError_t err2 = cudaGetLastError();
if (err2 != cudaSuccess) {
    printf("Error with name: %s\n", cudaGetErrorName(err2));
    printf("Error description: %s\n", cudaGetErrorString(err2));
}

if (err2 != err) {
    printf("As expected, cudaPeekAtLastError() did not clear the error\n");
}

// Check again
cudaError_t err3 = cudaGetLastError();
if (err3 == cudaSuccess) {
    printf("As expected, cudaGetLastError() cleared the error\n");
}
```

Tip

When an error appears at a synchronization, especially in a stream with many operations, it is often difficult to pinpoint exactly where in the stream the error may have occurred. To debug such a situation a useful trick may be to set the environment variable `CUDA_LAUNCH_BLOCKING=1` and then run the application. The effect of this environment variable is to synchronize after every single kernel launch. This can aid in tracking down which kernel, or transfer caused the error. Synchronization can be expensive; applications may run substantially slower when this environment variable is set.

2.3.5. CUDA Stream Ordering

Now that we have discussed the basic mechanisms of streams, events and callback functions it is important to consider the ordering semantics of asynchronous operations in a stream. These semantics are to allow application programmers to think about the ordering of operations in a stream in a safe way. There are some special cases where these semantics may be relaxed for purposes of performance optimization such as in the case of a *Programmatic Dependent Kernel Launch* scenario, which allows the overlap of two kernels through the use of special attributes and kernel launch mechanisms, or in the case of batching memory transfers using the `cudaMemcpyBatchAsync()` function when the runtime can perform non-overlapping batch copies concurrently. We will discuss these optimizations later on *link needed*.

Most importantly CUDA streams are what are known as in-order streams. This means that the order of execution of the operations in a stream is the same as the order in which those operations were enqueued. An operation in a stream cannot leap-frog other operations. Memory operations (such as copies) are tracked by the runtime and will always complete before the next operation in order to allow dependent kernels safe access to the data being transferred.

2.3.6. Blocking and non-blocking streams and the default stream

In CUDA there are two types of streams: blocking and non-blocking. The name can be a little misleading as the blocking and non-blocking semantics refer only to how the streams synchronize with the default stream. By default, streams created with `cudaStreamCreate()` are blocking streams. In order to create a non-blocking stream, the `cudaStreamCreateWithFlags()` function must be used with the `cudaStreamNonBlocking` flag:

```
cudaStream_t stream;
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
```

and non-blocking streams can be destroyed in the usual way with `cudaStreamDestroy()`.

2.3.6.1. Legacy Default Stream

The key difference between the blocking and non-blocking streams is how they synchronize with the **default stream**. CUDA provides a legacy default stream (also known as the NULL stream or the stream with stream ID 0) which is used when no stream is specified in kernel launches or in blocking `cudaMemcpy()` calls. This default stream, which was shared amongst all host threads, is a blocking stream. When an operation is launched into this default stream, it will synchronize with all other blocking streams, in other words it will wait for all other blocking streams to complete before it can execute.

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

kernel1<<<grid, block, 0, stream1>>>(...);
kernel2<<<grid, block>>>(...);
kernel3<<<grid, block, 0, stream2>>>(...);

cudaDeviceSynchronize();
```

The default stream behavior means that in the above code snippet above, *kernel2* will wait for *kernel1* to complete, and *kernel3* will wait for *kernel2* to complete, even if in principle all three kernels could execute concurrently. By creating a non-blocking stream we can avoid this synchronization behavior. In the code snippet below we create two non-blocking streams. The default stream will no longer synchronize with these streams and in principle all three kernels could execute concurrently. As such we cannot assume any ordering of execution of the kernels and should perform explicit synchronization (such as with the rather heavy handed `cudaDeviceSynchronize()` call) in order to ensure that the kernels have completed.

```
cudaStream_t stream1, stream2;
cudaStreamCreateWithFlags(&stream1, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);

kernel1<<<grid, block, 0, stream1>>>(...);
kernel2<<<grid, block>>>(...);
kernel3<<<grid, block, 0, stream2>>>(...);

cudaDeviceSynchronize();
```

2.3.6.2. Per-thread Default Stream

Starting in CUDA-7, CUDA allows for each host thread to have its own independent default stream, rather than the shared legacy default stream. In order to enable this behavior one must either use the *nvcc* compiler option `--default-stream per-thread` or define the `CUDA_API_PER_THREAD_DEFAULT_STREAM` preprocessor macro. When this behavior is enabled, each host thread will have its own independent default stream which will not synchronize with other streams in the same way the legacy default stream does. In such a situation the legacy default stream example will now exhibit the same synchronization behavior as the non-blocking stream example.

2.3.7. Explicit Synchronization

There are various ways to explicitly synchronize streams with each other.

`cudaDeviceSynchronize()` waits until all preceding commands in all streams of all host threads have completed.

`cudaStreamSynchronize()` takes a stream as a parameter and waits until all preceding commands in the given stream have completed. It can be used to synchronize the host with a specific stream, allowing other streams to continue executing on the device.

`cudaStreamWaitEvent()` takes a stream and an event as parameters (see CUDA Events for a description of events)and makes all the commands added to the given stream after the call to `cudaStreamWaitEvent()` delay their execution until the given event has completed.

`cudaStreamQuery()` provides applications with a way to know if all preceding commands in a stream have completed.

2.3.8. Implicit Synchronization

Two operations from different streams cannot run concurrently if any CUDA operation on the NULL stream is submitted in-between them, unless the streams are non-blocking streams (created with the `cudaStreamNonBlocking` flag).

Applications should follow these guidelines to improve their potential for concurrent kernel execution:

- All independent operations should be issued before dependent operations,
- Synchronization of any kind should be delayed as long as possible.

2.3.9. Miscellaneous and Advanced topics

2.3.9.1. Stream Prioritization

As mentioned previously, developers can assign priorities to CUDA streams. Prioritized streams need to be created using the **`cudaStreamCreateWithPriority()`** function. The function takes two parameters: the stream handle and the priority level. The general scheme is that lower numbers correspond to higher priorities. The given priority range for a given device and context can be queried using the **`cudaDeviceGetStreamPriorityRange()`** function. The default priority of a stream is 0.

```
int minPriority, maxPriority;

// Query the priority range for the device
cudaDeviceGetStreamPriorityRange(&minPriority, &maxPriority);

// Create two streams with different priorities
// cudaStreamDefault indicates the stream should be created with default flags
// in other words they will be blocking streams with respect to the legacy default stream
// One could also use the option `cudaStreamNonBlocking` here to create a non-blocking streams
cudaStream_t stream1, stream2;
cudaStreamCreateWithPriority(&stream1, cudaStreamDefault, minPriority);  // Lowest priority
cudaStreamCreateWithPriority(&stream2, cudaStreamDefault, maxPriority);  // Highest priority
```

We should note that a priority of a stream is only a hint to the runtime and generally applies primarily to kernel launches, and may not be respected for memory transfers. Stream priorities will not preempt already executing work, or guarantee any specific execution order.

2.3.9.2. Introduction to CUDA Graphs with Stream Capture

CUDA streams allow programs to specify a sequence of operations, kernels or memory copies, in order. Using multiple streams and cross-stream dependencies with `cudaStreamWaitEvent`, an application can specify a full directed acyclic graph (DAG) of operations. Some applications may have a sequence or DAG of operations that needs to be run many times throughout execution.

For this situation, CUDA provides a feature known as CUDA graphs. This section introduces CUDA graphs and one mechanism of creating them called *stream capture*. A more detailed discussion of CUDA graphs is presented in CUDA Graphs. Capturing or creating a graph can help reduce latency and CPU overhead of repeatedly invoking the same chain of API calls from the host thread. Instead, the APIs to specify the graph operations can be called once, and then the resulting graph executed many times.

CUDA Graphs work in the following way:

i. The graph is *captured* by the application. This step is done once the first time the graph is executed. The graph can also be manually composed using the CUDA graph API.

ii. The graph is *instantiated*. This step is done one time, after the graph is captured. This step can set up all the various

Last updated on Mar 04, 2026.